🏏

DBMS Unit - 2 (Part 1)

📘 Relational Model Concepts

🧩 1. Overview

Relational Model of Data

- Based on the **concept of a Relation** (like a mathematical relation).
- Foundation of modern databases (formalized by E. F. Codd).
- Provides a **formal, structured** way to organize data into **tables**.

👉 **Key Strength:**

It's grounded in **mathematical theory of relations**, giving it a strong logical base.

🧠 2. Informal Definitions

A **relation** ≈ **a table of values**

Concept	Informal View	Example
Rows	Represent facts about entities or relationships	Each student record
Columns	Represent attributes (properties)	Name, SSN, Age, GPA
Table name	Represents a relation name	STUDENT
Cell value	Data item from domain	"Benjamin Bayer"

🧍 Rows = Tuples

- Each **row** = **tuple**
- Represents a single record or fact
- Example:
  `<"Benjamin Bayer", 305-61-2435, 19, 3.21>`

📊 Columns = Attributes

- Each **column** has a **name (attribute)**
- Defines the **meaning** of data in that column
- Example: "SSN" or "Address"

🔑 3. Key of a Relation

Type	Description	Example
Primary Key	Attribute(s) that **uniquely identify** a row	SSN in STUDENT
Artificial / Surrogate Key	A generated key (like a Row ID) used when no natural key exists	Student_ID = 1, 2, 3...

🧭 **Rule:** No two tuples in a relation have the same key value.

🧾 4. Formal Definitions

🧱 (a) Relation Schema

Defines the **structure** of a relation.

Syntax:

`R(A1, A2, ..., An)`

Where:

- `R` = Relation name
- `A1, A2, ..., An` = Attributes

Example:

`CUSTOMER(Cust_id, Cust_name, Address, Phone#)`

📋 (b) Tuple

- An **ordered set of values** → `<v1, v2, …, vn>`
- Each value comes from a **domain**

Example:

`<632895, "John Smith", "101 Main St", "(404) 894-2000">`

→ a 4-tuple in the CUSTOMER relation

🌐 (c) Domain

Defines **set of valid values** for an attribute.

Attribute	Domain Example	Format
`Cust-id`	6-digit number	000000–999999
`Phone#`	USA phone numbers	(ddd)ddd-dddd
`Date`	Calendar dates	yyyy-mm-dd

🧩 One domain can be used in different attributes:

e.g., `Date` → used as *Invoice-date*, *Payment-date*.

🧮 (d) Relation State

- A **subset of the Cartesian product** of attribute domains.
- Represents the **current content** (data) of the relation.

Formally:

If `R(A1, A2, …, An)` →

then `r(R) ⊆ dom(A1) × dom(A2) × … × dom(An)`

🧠 Each row (tuple) = `<v1, v2, …, vn>` where `vj ∈ dom(Aj)`

🧩 5. Example Walkthrough

📘 Relation Schema:

`STUDENT(Name, SSN, Home_phone, Address, Office_phone, Age, GPA)`

📊 Relation State:

Name	SSN	Home_phone	Address	Office_phone	Age	GPA
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Ln	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Rd	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Rd	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Ln	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Ln	NULL	19	3.25

Here:

- **Relation Name:** STUDENT
- **Attributes:** Name, SSN, Home_phone, etc.
- **Tuples:** Each row (5 total)
- **Key:** SSN
- **NULL:** Missing or unknown values

🔍 6. Example of Formal Relation

Let `R(A1, A2)` be a relation schema

- `dom(A1) = {0,1}`
- `dom(A2) = {a,b,c}`

Then:

`dom(A1) × dom(A2) = {<0,a>, <0,b>, <0,c>, <1,a>, <1,b>, <1,c>}`

Possible relation state:

`r(R) = {<0,a>, <0,b>, <1,c>}`

→ This is one valid population (state) of R.

📚 7. Summary Table

Informal Term	Formal Term	Example
Table	Relation	STUDENT

Informal Term	Formal Term	Example
Column Header	Attribute	Age
All possible column values	Domain	dom(Age) = {0–100}
Row	Tuple	`<Rohan Panchal, 489-22-1100, 28, 3.93>`
Table Definition	Schema	STUDENT(Name, SSN, GPA)
Populated Table	State	Actual data (rows)

📘 Characteristics of Relations & Integrity Constraints

🧩 1. Characteristics of Relations

A **relation** = a **table** in a database.

Each relation has **rows (tuples)** and **columns (attributes)**.

🧾 1.1 Ordering of Tuples (Rows)

- The **rows in a table** have **no fixed order**.
- Even if they appear in a sequence, it **does not matter**.
- You can change the order of rows → **it's still the same relation**.

✅ *Example:*

In the STUDENT table, swapping Rohan and Barbara's rows doesn't change the meaning.

📑 1.2 Ordering of Attributes (Columns)

- **Columns are ordered** based on the schema.
- In every tuple (row), values must follow that same order.

✅ *Example:*

If relation schema is

`STUDENT(Name, SSN, Address, GPA)`

→ tuple must follow this order

`<"Benjamin Bayer", 305-61-2435, "2918 Bluebonnet Ln", 3.21>`

⚛️ 1.3 Atomic (Single-Valued) Attributes

- Each value in a table must be **atomic** → can't be divided further.
- Multi-valued or composite data is **not allowed** in a relation.

✅ *Example:*

✔️ Valid: City = "Pune"

❌ Invalid: City = {"Pune", "Mumbai"}

🧮 1.4 Domain of Attributes

- Each column (attribute) has a **domain** → a set of **valid values** it can take.
- All values in that column must belong to its domain.

✅ *Example:*

- Domain of "Age" = 0 to 120
- Domain of "Phone" = 10-digit numbers

🚫 1.5 NULL Values

- **NULL** = value is *unknown* or *not applicable*.
- Used when data is missing or doesn't apply to that tuple.

✅ *Example:*

Some students have `Office_phone = NULL` → means they don't have one.

🧠 1.6 Tuple Notation

- We can write:
  - `t[Ai]` or `t.Ai` → value of attribute *Ai* in tuple *t*
  - `t[Name, Age]` → values of Name and Age together (subtuple)

🔐 2. Relational Integrity Constraints

These are **rules** that must always be true for the database to stay consistent.

⚙️ 2.1 Types of Constraints

Type	Meaning	Example
Domain Constraint	Value must come from the defined domain	Age = 25 ✅, Age = "blue" ❌
Key Constraint	No two rows can have the same key value	No two students can have same SSN
Entity Integrity	Primary key cannot be NULL	SSN must always have a value
Referential Integrity	Foreign key value must match a valid primary key in another table	Employee's Dno must exist in Department table

---

🧩 3. Key Constraints (Detailed)

Keys are used to **uniquely identify** each row in a table.

🗝️ 3.1 Superkey

- Any **set of attributes** that can uniquely identify a row.
- A table can have many superkeys.

✅ *Example:*

`CAR(LicenseNumber, EngineSerialNumber, Make, Model, Year)`

Superkeys:

- {LicenseNumber}
- {EngineSerialNumber}
- {LicenseNumber, Make}

🔑 3.2 Key (Minimal Superkey)

- A **superkey with no extra attributes**.
- If you remove any attribute, it will no longer be unique.

✅ *Example:*

In CAR table:

- {LicenseNumber} → key
- {EngineSerialNumber} → key
- {LicenseNumber, Make} → not minimal → not a key

🧾 3.3 Candidate Keys

- All possible **minimal superkeys** are called *candidate keys*.

✅ *Example:*

CAR table has two candidate keys:

- {LicenseNumber}
- {EngineSerialNumber}

🌟 3.4 Primary Key

- One candidate key is **chosen as the main key** → *primary key*.
- Used to **identify rows** and create relationships.
- **Cannot contain NULL** values.

✅ *Example:*

CAR table → {LicenseNumber} chosen as Primary Key.

🔍 3.5 Superkey vs Key Summary

Concept	Meaning	Example
Superkey	Any set of attributes that uniquely identify a row	{LicenseNumber, Make}
Key	Minimal Superkey	{LicenseNumber}
Candidate Keys	All possible minimal keys	{LicenseNumber}, {EngineSerialNumber}
Primary Key	One chosen key	{LicenseNumber}

🧱 4. Relational Database Schema

A **database schema** = collection of all relation schemas in the database.

📘 **Notation:**

`S = {R1, R2, ..., Rn}`

→ each Ri is a relation (table).

🏢 Example: COMPANY Database Schema

Relation	Attributes
EMPLOYEE	Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno
DEPARTMENT	Dname, Dnumber, Mgr_ssn, Mgr_start_date
DEPT_LOCATIONS	Dnumber, Dlocation
PROJECT	Pname, Pnumber, Plocation, Dnum
WORKS_ON	Essn, Pno, Hours
DEPENDENT	Essn, Dependent_name, Sex, Bdate, Relationship

🧭 5. Quick Recap

Concept	Description	Example
Relation	Table of data	STUDENT
Tuple	Row in a table	<"Rohan Panchal", 489-22-1100, 28, 3.93>
Attribute	Column name	Age, GPA
Domain	Valid values for a column	Age → 0–120
Key	Uniquely identifies a row	SSN
NULL	Missing or inapplicable data	Office_phone = NULL

🧩 Entity Integrity

🧱 What it means:

- Every **relation (table)** must have a **Primary Key (PK)** that **uniquely identifies each row (tuple)**.
- **Primary key values cannot be NULL** — because if they were NULL, we wouldn't be able to identify that row.

✅ **Rule:**

For every tuple `t` in relation `r(R)`,

`t[PK] ≠ NULL`

⚙️ If the primary key has multiple attributes:

- **None** of them can be NULL.
- Together, they must form a complete unique identifier.

💡 Example:

`EMPLOYEE(Ssn, Name, Salary, Dno)`

- `Ssn` = Primary Key

- So every employee must have a **non-null SSN**.

❌ Invalid:

`<NULL, John Smith, 50000, 5>` → violates entity integrity

✔️ Valid:

`<123456789, John Smith, 50000, 5>`

📝 Note:

Other attributes (like *Address* or *Salary*) **can be NULL**, if allowed —

but **not the Primary Key**.

🔗 Referential Integrity

🧭 What it means:

- This rule **connects two tables** through a **Foreign Key (FK)**.
- It ensures that **values in the FK column of one table** must **match a Primary Key value** in another table — or be **NULL**.

💬 Example:

`EMPLOYEE(Dno)` → references → `DEPARTMENT(Dnumber)`

This means:

- Each employee's `Dno` value must match an existing department's `Dnumber`.
- If `Dno = 5`, there must be a department with `Dnumber = 5`.

🔐 Terms:

Term	Meaning
Referencing Relation (R1)	The table that has the foreign key
Referenced Relation (R2)	The table that has the primary key
FK → PK	FK values must match PK values in another table

✅ Example:

If `R1 = EMPLOYEE` and `R2 = DEPARTMENT`:

- EMPLOYEE's `Dno` is a **foreign key**
- DEPARTMENT's `Dnumber` is a **primary key**

📘 Formal Statement:

The foreign key value in R1 can be:

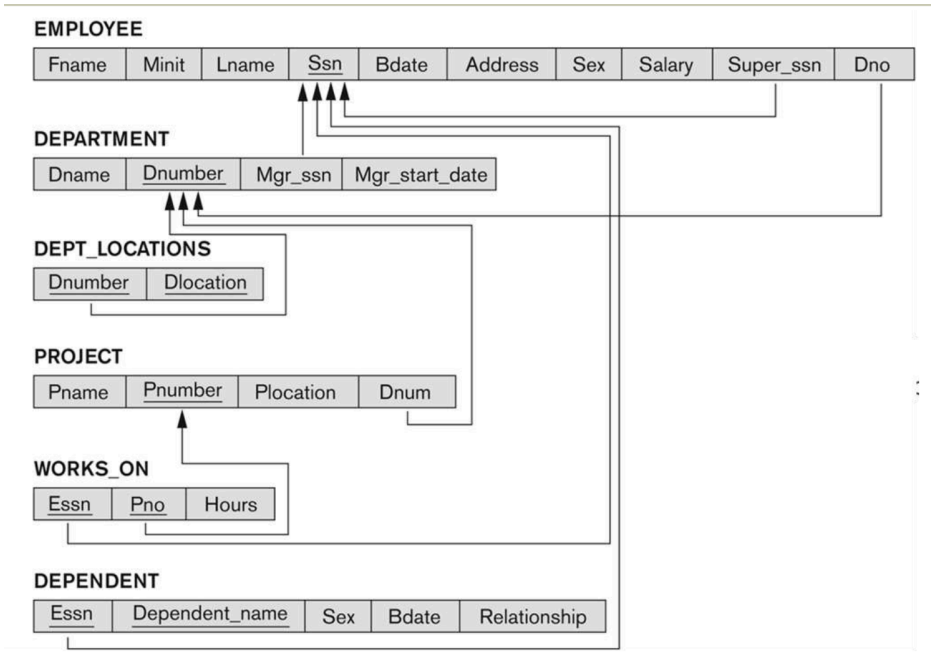
1. A **value** that already exists as a **primary key in R2**, or
2. **NULL** (if that attribute is not part of the primary key).

🔁 Diagram Representation:

In schema diagrams:

- Primary keys are **underlined**
- A **foreign key** is shown as an **arrow** from R1.FK → R2.PK

✅ Example (COMPANY Database):



🧩 Other Constraints

🧠 Semantic Integrity Constraints

- These are **rules based on business logic** or **real-world meaning** — not just table structure.
- They cannot always be defined using standard SQL constraints.

✅ Example:

> "An employee cannot work more than 56 hours per week across all projects."

📘 Tools to define these:

- **Triggers**
- **Assertions** (in SQL-99 and later)

🧱 4. Database State

💾 What it means:

- The **current content** of all tables in a database at a given time.

- Each table has many **tuples** = its **relation state**.
- All relation states together = **database state**.

⚙️ Changes in database state happen through:

1. **INSERT** → Add a new row
2. **DELETE** → Remove an existing row
3. **MODIFY (UPDATE)** → Change data in a row

Each operation must **preserve all integrity constraints**.

🔄 5. Update Operations and Integrity

🧩 5.1 INSERT

When adding a new row, it might violate:

Constraint Type	When it's violated
Domain constraint	Value doesn't match the attribute's data type or domain
Key constraint	Duplicate primary key value
Referential integrity	Foreign key points to a non-existing value
Entity integrity	Primary key is NULL

🗑️ 5.2 DELETE

When removing a tuple, it may violate:

- **Referential integrity**, if the deleted primary key is **referenced by another table**.

✅ Example:

Deleting a DEPARTMENT with Dnumber=5

→ violates constraint if EMPLOYEE.Dno=5 exists.

🧭 Options to handle this:

Option	Meaning
RESTRICT / REJECT	Stop deletion
CASCADE	Delete all dependent rows automatically
SET NULL	Set foreign keys in other tables to NULL

✏️ 5.3 UPDATE

When changing values:

Update Type	Possible Violation
Update PK	Acts like DELETE + INSERT → may break FKs
Update FK	May violate referential integrity
Update normal attribute	May violate domain constraint

🧩 6. COMPANY Example (Summary)

Relation	Primary Key	Foreign Key
EMPLOYEE	Ssn	Super_ssn, Dno
DEPARTMENT	Dnumber	Mgr_ssn
DEPT_LOCATIONS	(Dnumber, Dlocation)	Dnumber
PROJECT	Pnumber	Dnum
WORKS_ON	(Essn, Pno)	Essn, Pno
DEPENDENT	(Essn, Dependent_name)	Essn

📊 Arrows (FK → PK):

- EMPLOYEE.Dno → DEPARTMENT.Dnumber
- WORKS_ON.Essn → EMPLOYEE.Ssn
- DEPENDENT.Essn → EMPLOYEE.Ssn
- PROJECT.Dnum → DEPARTMENT.Dnumber

✅ 7. Quick Recap Table

Concept	Rule / Meaning	Example
Entity Integrity	Primary key cannot be NULL	Ssn must not be NULL
Referential Integrity	FK must match a PK in another table or be NULL	Dno must exist in DEPARTMENT
Semantic Constraint	Based on real-world logic	Max 56 work hours per week
INSERT	Adds a row	Must not violate any constraint
DELETE	Removes a row	May break FK reference
UPDATE	Modifies data	May break PK, FK, or domain rule

DO ER→RELATIONAL MODEL PART FROM HERE (VIP)